INF1103: Lab 7

Object-Oriented Relationships Part 1 (Inheritance, Overriding, Abstract, Interfaces)

Introduction

In Lab 6, you refactored the Inventory Management System using object-oriented programming. You created a Product class to represent products and an Inventory class to manage them.

However, different products may have different characteristics. For example, a physical product needs to be stored and shipped, while a digital product may be delivered through a download link. This gives us an opportunity for Inheritance. Remember the 'is-a' relationship we talked about in the lecture. Always say the sentence and ensure it makes sense! So, a guitar 'is-a' physical product, an operating system 'is-a' digital product. The sentences work, make logical sense, so we have inheritance!

In this lab, you will extend your existing system using:

- Inheritance
- Method overriding
- Abstract classes
- Interfaces using abstract base classes

You will continue to apply encapsulation and the SOLID principles with the 'S' Single Responsibility Principle (SRP) and we will start to expand more on the O in relation to the Open Close Principle.

Learning outcomes

By the end of this lab, you should be able to:

1. Explain and implement inheritance in Python.
2. Override inherited methods to customise behaviour.
3. Create abstract classes using ABC and @abstractmethod.
4. Define an interface-style abstract base class that specifies required behaviour.
5. Update an existing system to support different product types.

1. Extend the Product class using inheritance

1.1 Review your existing implementation

Open your Inventory Management System from Lab 6.

Your existing Product class should contain attributes such as Product ID, Name, Price and Stock quantity. It should also provide methods for updating stock and displaying product information. This is suitable for forming the base of other types of products.

Ideally once the product class has been created, it should not need to be modified when a new product is introduced and instead specialized products (Digital / Physical products) would have their own classes which inherit (Extend) from the product class with their own attributes and behaviors.

This is a nice example of Open-Close Principle (OCP): Software Entities should be open for extension but closed for modification.

Note-1: This doesn't mean we can never modify product, but here modification should be related to fixing or enhancing something that product is solely responsible for, not trying to extend functionality.

Note-2: While inheritance supports OCP, it just one approach and there are others that can support OCP (so inheritance isn't the only option).

So, lets extend the functionality using product as our base and work on physical and digital product classes!

1.2 Create a PhysicalProduct class

inheritance
physicalproduct.py

Create a new class called PhysicalProduct that inherits from Product.

Considering this is a physical item it should retain the attributes and behaviour of a regular product while introducing new attributes and behaviours that are unique to a physical product, let's add weight!

- weight: the weight of the product in kilograms.

this is most likely in float, because 4.45kg is just as valid as 29000kg

Your implementation should:

6. Reuse the existing Product constructor using super().
7. Initialise the additional weight attribute.
8. Provide a method called display_information() that displays the product's existing information alongside its weight.

as typical in a child class

weight

Test your class by creating a PhysicalProduct object from your external code (where you run your program!) and displaying its information.

main.py?

1.3 Create a DigitalProduct class

Create another class called DigitalProduct that inherits from Product.

A digital product exists in the virtual realm, so doesn't actually have a weight, but it does have a download size, which is a suitable new attribute to be added, so now let's add download_size:

- download_size: the size of the downloadable product in megabytes.

also a float

Your implementation should:

9. Reuse the existing Product constructor using super().
10. Initialise the additional download_size attribute.
11. Provide its own display_information() method that displays the product's existing information alongside its download size.

Test your class by creating a DigitalProduct object and displaying its information.

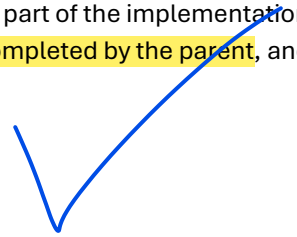
2. Method overriding

Currently based on the above implementation you'll notice that both PhysicalProduct and DigitalProduct provide their own implementations of display_information(). This is an example of method overriding where a subclass provides its own implementation of a method inherited from its parent class. The flow is straight forward, the child implementation will always take preference over the parents.

2.1 Reuse existing behaviour

all the behaviours of the parent class, but with new attributes initialised in the child classes

Update the display_information() method in each subclass to call the parent implementation using super() before displaying the additional information. The subclass should reuse the existing product information rather than duplicating all the code from Product. This means, part of the implementation of display_information (the part that uses common attributes from the parent) will be completed by the parent, and the child will add on the parts in display_information that are unique to the child.



Then we integrate this to main.py

2.2 Test your implementation

Create one physical product and one digital product. Call display_information() on each object and check that the output contains:

- The common product information provided from the parent class.
- The additional information specific to that product type provided by the appropriate child class.

final step is main.py integration

3. Abstract classes

Currently, it is possible to create a generic Product object by itself, this would mean a product that is not physical or digital. As a software designer, we have decided that every product will be either digital and physical, so having a product which is not one or the other would not be suitable. A physical product might be shipped, while a digital product might be downloaded.

We can use an abstract class to define behaviour that subclasses must implement, and by making a class truly abstract (inherit ABC and add @abstractMethod in python), then that class by itself will never be allowed to be instantiated directly. Product..... will crash the code

3.1 Convert Product into an abstract class this is where we integrate the 2 child classes to main.py

Import the required functionality from Python's abc module:

```
from abc import ABC, abstractmethod
```

Update Product so that it inherits from ABC then add an abstract method called get_fulfilment_info().

This method should be decorated with @abstractmethod and should not provide a concrete implementation.

@abstractmethod signifies an abstract function

3.2 Implement the required method

this is the abstract function

Update both PhysicalProduct and DigitalProduct to implement get_fulfilment_info(). For example:

- PhysicalProduct returns a message explaining that the product requires physical shipping.
- DigitalProduct returns a message explaining that the product is delivered digitally.

Test both implementations to ensure they are behaving correctly. still cannot get this to work

3.3 Test the abstract class

Attempt to create a Product object directly. Observe what happens when you try to instantiate the class while its abstract method remains unimplemented. it will crash because you cannot instantiate an abstracted class

Note: An abstract class can contain both regular methods and abstract methods, regular methods can still provide implemented behaviours that the sub (child) classes can use, while abstract method will provide no implementation and force the child to complete that implementation themselves.

4. Interfaces using abstract base classes

As we know, not all class relationships fall into the 'is-a' category. For example let's say we want to group objects into those that CanBeShipped, this is a 'mustbeableto' relationship. So we could not say 'a guitar **is-a** shipped', but would could say, a guitar **mustbeableto** be shipped', so yes, it CanBeShipped. Classes need to support a particular capability without necessarily sharing the same parent class. In this example some products may support shipping, while others do not.

Python does not have a dedicated interface keyword. Instead, an abstract base class can be used to define an interface-style contract: the methods a class must provide to support a particular capability. If the interface has a

CanBeShipped abstract method defined, then any class that implements the interface (signs the contract) must be able to deliver on that functionality, and must implement it.

4.1 Create a CanBeShipped interface

Create a new abstract base class called CanBeShipped.

because this is interface, we do not need __init__

It should define an abstract method called calculate_shipping_cost(). The method should specify that implementing classes must provide a way to calculate shipping costs.

Do not add product-specific attributes or shipping implementation details to this interface. The interface is a contract that should be signable by any object that can-be-shipped.

4.2 Implement the interface

Update PhysicalProduct so that it implements the CanBeShipped interface. Implement calculate_shipping_cost() using a simple rule of your choice. For example, calculate shipping based on the product's weight.

Test the method with physical products of different weights.

so we just make the changes on DigitalProduct itself

Do not make DigitalProduct implement CanBeShipped, as digital products do not require physical shipping.

Important: Remember to stick with single class inheritance. PhysicalProduct should inherit from Product, and CanBeShipped should remain a separate interface-style ABC. Avoid multiple inheritance relationships, but a single inheritance and multiple interface relationships is ok!

5. Integrate your classes into the Inventory Management System

Ok, let's tie all of this together with your Inventory Management System. Update your system to support the physical and digital product setup that we just established.

Your system should continue to support the functionality from Lab 6:

- Display all products.
- Add a product.
- Update stock.
- Search for a product.
- Save and load inventory data.
- Exit the application.

saves first and then exit

When adding a product, the user should be allowed to choose between a physical product and a digital product and enter the additional information required for that product type as defined in physical and digital product classes.

Ensure that your Inventory class continues to manage product objects, and doesn't implement their behaviours (of course it should call their methods by not 'DO' their methods or methods that should belong to the class)

JSON persistence

JSON? I thought it's dictionary

Update your JSON saving and loading logic to preserve the product type and its additional attributes. For example, each saved product should include a type identifier such as "physical" or "digital".

When loading the data, use that identifier to create the appropriate subclass. Test the system by saving an inventory containing both product types, restarting the program, and verifying that the correct product information is restored.

Ohh, I somehow forgot I was using JSON all along

6. UML class diagram

Update your UML class diagram to represent the revised system. Remember, details of this are available from our Lecture on Monday with a 'walked through' view from code to Class Diagram. Be sure to Include:

- Product as an abstract class.
- PhysicalProduct and DigitalProduct as subclasses of Product.
- CanBeShipped as an interface-style abstract base class.
- Inventory and its relationship with Product.

Use the appropriate UML notation:

- Inheritance/generalisation: solid line with a hollow triangle pointing towards the parent class.
- Interface realisation: dashed line with a hollow triangle pointing towards the interface.
- Abstract classes and abstract methods: use <<abstract>> to define an abstract class, according to the notation used during the lecture session.

7. Testing and review

Test your implementation to confirm that:

- Physical products store and display their weights.
- Digital products store and display their download sizes.
- Both subclasses implement get_fulfilment_info().
- The abstract Product class cannot be instantiated directly.
- Physical products can calculate shipping costs.
- Digital products are not required to implement shipping behaviour.
- Inventory operations continue to work.
- JSON saving and loading preserve the correct product types.

does Product.display_goodbye_message() still count?

Review your code for opportunities to reuse existing behaviour and keep responsibilities separated.

8. Reflection Task (Optional)

Yes, you read that correctly, this is an optional task! This is more about reflection but would help you understand both Inheritance and OCP better.

- Add a new product type called SubscriptionProduct to your InventoryManagementSystem
- Ideally you should be able to do this without changing the existing Product class or modifying any product handling code.
- Reflection: How well does your current design support adding a new product type? Which parts of the existing system needed to change? How could the design be enhanced to reduce or remove those required changes when adding a new product type? (hint: Not an inheritance solution in this case! Might be tricky, hence the task is optional!)

I made some integration changes involving all 3 classes. Including moving them to a products folder

How well does the design support a new type? The original inheritance-based design handled the class itself easily, but the type-specific knowledge was scattered across the menu and FileHandling as if/elif chains and isinstance checks. Each new type would have meant editing all of them, which violates OCP.

9. Managing Project Files with .gitignore

As your object-oriented project grows, you might noticed that more and more files are being created. Rather than adding each one to git individually, git allows you to stage changes together. The .gitignore file helps prevent unnecessary files (such as cache files, virtual environments etc.) from being added in your repository.

git add .

- Create a .gitignore file in your project root (Can be done in VSCode by clicking top level project folder, create a new file named .gitignore.
- Add the appropriate exclusions, a sample may be (remember these are things which are excluded, all else is included)

```
# Python cache files
__pycache__/
*.py[cod]

# Virtual environments
.venv/
venv/

# Local environment variables
.env

# IDE settings
.vscode/
.idea/
```

- Use git status to inspect your working tree
- Stage your project changes using git add . (don't forget the dot!), and inspect the changes. This basically adds all new and modified files to staged except those that are 'ignored'.
- Commit your changes with a meaningful message
- Push changes to the remote repository.
- Run your program and tests using the established Docker workflow.
- Review your commit history using git log.

Submission requirements

Submit your updated Inventory Management System, including:

- Python source files.
- Updated UML class diagram.
- JSON data file or other test data used to demonstrate persistence.
- Screenshot of git and docker workflow as per previous labs.

Looking ahead

In the next lab, you will explore association, aggregation and composition to model relationships between objects. You will then build on your understanding of inheritance and method overriding when studying polymorphism.